

Lab 01 Report

Q1

Algo	I	J	K	avg_time (s)
matmul_ijk	256	256	256	0.028859
matmul_ijk	256	256	512	0.064137
matmul_ijk	256	512	256	0.063693
matmul_ijk	256	512	512	0.242668
matmul_ijk	512	256	256	0.064273
matmul_ijk	512	256	512	0.172387
matmul_ijk	512	512	256	0.127140
matmul_ijk	512	512	512	0.296135
matmul_ikj	256	256	256	0.018771
matmul_ikj	256	256	512	0.034708
matmul_ikj	256	512	256	0.033565
matmul_ikj	256	512	512	0.069985
matmul_ikj	512	256	256	0.036108
matmul_ikj	512	256	512	0.069444
matmul_ikj	512	512	256	0.066929
matmul_ikj	512	512	512	0.128594
matmul_AT	256	256	256	0.031184
matmul_AT	256	256	512	0.066480
matmul_AT	256	512	256	0.064013
matmul_AT	256	512	512	0.133887
matmul_AT	512	256	256	0.063163
matmul_AT	512	256	512	0.134656
matmul_AT	512	512	256	0.129534

Algo	I	J	K	avg_time (s)
matmul_AT	512	512	512	0.308100
matmul_BT	256	256	256	0.032687
matmul_BT	256	256	512	0.069032
matmul_BT	256	512	256	0.066508
matmul_BT	256	512	512	0.137129
matmul_BT	512	256	256	0.065552
matmul_BT	512	256	512	0.130577
matmul_BT	512	512	256	0.124515
matmul_BT	512	512	512	0.254247

分析

从结果可以看出，`matmul_ikj` 明显快于 `matmul_ijk`，`C[i][j]`，`B[k][j]` 在 `ikj` 中随着 `j` 变化连续访问，cache 命中率更高。`ijk` 中最内层是 `k`，导致 `B[k][j]` 跨行访问，速度降低。

`ijk` 的访问模式下，`matmul_AT` 内层 `k` 循环访问的是 `AT[k][i]` 和 `B[k][j]`，都存在跨行访问。`matmul_BT` 里内层 `k` 循环访问的是 `A[i][k]` 和 `BT[j][k]`，cache 命中率更高，理论上更快。但实际测试中差距并不明显，因为转置存在开销，BT相比于AT的优势需要在更大的矩阵乘法下才能得到体现。同时，转置后的两个版本都比 `matmul_ikj` 慢，因为多出了转置开销。

Q2

实现位于 [Lab1-GEMM-1/code/matmul.cpp](#)。

卷积配置为：

- `batch = 1`
- `height = 56`
- `width = 56`
- `in_channels = 3`
- `out_channels = 64`
- `kernel = 3`
- `stride = 1`
- `padding = 0`

计算流程：

1. 使用 `im2col` 将输入张量展开为二维矩阵。

- 2. 将卷积核权重 reshape 为二维矩阵。
- 3. 将卷积计算转化为矩阵乘法计算得到结果。

Q3

当前实现了四种单独优化，以及一个组合优化版本 `hybrid_fast`。

优化说明

`loop_unrolling`：

- 将 `j` 方向按 4 个元素做循环展开。

`writing_caching`：

- 通过局部 `sum` 累加，最后一次性写回 `c[i][j]`，减少对同一输出元素的反复读写。

`tiling`：

- 将 `I/J/K` 分块处理，块大小目前为32。

`vectorization`：

- 使用 `vector_size(16)` 的 4 整数向量类型，一次处理 4 个输出元素。

`hybrid_fast`：

- `tiling`：先把 `A/B/C` 按 `32 x 32` 分块，缩小工作集
- `array packing`：把当前 `B` 子块拷到连续缓冲区 `Bblk`
- `SIMD`：用 `v4i` 一次处理 4 个 `int`
- `loop unrolling`：`j` 方向按 4 个元素一组处理

单项优化结果

Algo	I	J	K	avg_time (s)
loop_unrolling	256	256	256	0.016383
loop_unrolling	256	256	512	0.036405
loop_unrolling	256	512	256	0.030442
loop_unrolling	256	512	512	0.061960
loop_unrolling	512	256	256	0.031164
loop_unrolling	512	256	512	0.061301

Algo	I	J	K	avg_time (s)
loop_unrolling	512	512	256	0.063570
loop_unrolling	512	512	512	0.124170
writing_caching	256	256	256	0.011040
writing_caching	256	256	512	0.019922
writing_caching	256	512	256	0.026155
writing_caching	256	512	512	0.049905
writing_caching	512	256	256	0.023036
writing_caching	512	256	512	0.044752
writing_caching	512	512	256	0.049129
writing_caching	512	512	512	0.100346
tiling	256	256	256	0.019495
tiling	256	256	512	0.041048
tiling	256	512	256	0.034600
tiling	256	512	512	0.067299
tiling	512	256	256	0.033447
tiling	512	256	512	0.068097
tiling	512	512	256	0.089480
tiling	512	512	512	0.150088
vectorization	256	256	256	0.006175
vectorization	256	256	512	0.010622
vectorization	256	512	256	0.015648
vectorization	256	512	512	0.025664
vectorization	512	256	256	0.011246
vectorization	512	256	512	0.023058
vectorization	512	512	256	0.027799
vectorization	512	512	512	0.051073

组合优化结果

Algo	I	J	K	avg_time (s)
hybrid_fast	256	256	256	0.004832
hybrid_fast	256	256	512	0.011659
hybrid_fast	256	512	256	0.007989
hybrid_fast	256	512	512	0.021003
hybrid_fast	512	256	256	0.009384
hybrid_fast	512	256	512	0.016941
hybrid_fast	512	512	256	0.015838
hybrid_fast	512	512	512	0.038985

分析

从单项优化结果来看，vectorization 的收益最明显，显著提高了指令级并行。writing_caching 表现其次，减少了对输出矩阵的反复读写。tiling 效果再次，提升缓存命中率。loop_unrolling 的提升相对有限，因为它主要减少循环计数和分支开销，并没有从根本上改变访存模式。

组合优化 hybrid_fast 的结果明显优于单项优化。hybrid_fast 先用 32 x 32 分块，再把当前 B 子块打包到连续缓冲区 Bblk，让块内读连续；随后在块内按 4 个元素一组进行向量化乘加，同时保留循环展开。测试效果明显快于最佳单项优化 vectorization 的 0.051073 s，也大幅优于 writing_caching 和 tiling。这说明单独优化某一个环节只能缓解局部瓶颈，而组合优化才能同时改善数据流、存储局部性和计算吞吐，因此在本实验中表现最好。